

A. SERIALIZATION

This refers to converting data structures or objects states into a format that can be stored or transmitted and reconstructed later.

YAML FOR CONFIGURATION

- Human data readable serialization language
YAML uses simple indentation instead of many brackets or symbols. This makes configuration files cleaner and easier to understand.
- Commonly used for configuration files and data exchange.
It provides a simple and human-readable way to store and share structured data between systems and applications.
- Supports complex data structures with a simple syntax
Supports complex data structures such as lists, dictionaries, and nested objects while still using a simple and easy-to-read syntax based mainly on indentation.
- YAML has low verbosity because it uses fewer characters compared to formats like XML.
- YAML support human readability because it uses simple indentation instead of many brackets or symbols

PROTOBUF ON THE SENSOR-TO-SERVER LINK

- **Schema support:**
Protobuf uses a `.proto` file to define the structure and data types of telemetry messages. This ensures both the sensors and the server interpret readings consistently and reduces data format errors.
- **High performance:**
Encoding and decoding Protobuf messages is faster than parsing text-based formats such as XML or JSON. This helps the AsyncIO server process many concurrent sensor connections efficiently

JSON/XML ON THE PUBLIC REST API

- Easy to read and write for humans.
- Easy to parse and generate for machines.
- Commonly used in web APIs and configurations.

B. WEB SERVICES PROTOCOL

- REST is the architecture style for designing networked applications.
- **Uses standard HTTP methods:**
`GET`, `POST`, and `DELETE` naturally match retrieving, adding, and removing sensor data.
- Stateless communication
- REST is stateless because each request includes all required data and the server doesn't store session information, making it easier to scale. SOAP can

be stateful or stateless, but often keeps session data between requests, adding complexity and overhead. REST is resource-oriented, meaning it focuses on resources like sensors and readings that are accessed through URLs (e.g., `/sensors/{id}`). SOAP is not resource-oriented; it is operation-oriented, where communication is based on calling specific actions or methods rather than interacting with resources.

- REST is stateless because each request is independent and carries all needed data, so the server doesn't store session information. RPC is more function-based and often relies on tighter client-server interaction, making it less naturally stateless than REST. REST is resource-oriented, meaning it works with resources like sensors and readings accessed through URLs. RPC is operation-oriented, focusing on calling specific remote functions or procedures rather than working directly with resources.

C. ASYNCIO VS THREADING

- This is an **I/O-bound problem** because the program spends most of its time waiting for operations such as network communication, file access, or database responses instead of performing heavy CPU calculations. AsyncIO is preferred because it uses coroutines (`async` and `await`) and an event loop to handle many tasks efficiently within a single thread.
- When one task is waiting for I/O, the event loop switches to another task without blocking the program. This makes AsyncIO more lightweight and scalable than threading, which uses more memory and can introduce synchronization issues. Threads are better for blocking libraries or background tasks, while processes are better for CPU-bound work such as machine learning or video processing.

D. HTTP MECHANISMS

We picked **content negotiation** and **cookies** since they directly affect REST API design.

- **Content negotiation:**
The API uses the `Accept` header to decide whether to return JSON, XML, or YAML. This lets different clients (mobile apps, scripts, legacy tools) use the same endpoint but receive data in a format they understand.
- **Cookies:**
Cookies are used to identify client sessions on the REST API. This allows the server to track clients across multiple requests (e.g., for session-based access control or personalized behavior) without requiring authentication data in every request body.

REFERENCES

NET322: Network Programming & Applications Development. *Serialization & Web Services: JSON, XML, YAML, Protobuf, REST, SOAP, RPC*. Lecture notes, University of Malawi.

NET322: Network Programming & Applications Development. *HTTP Deep Dive: Programming for the Web*. Lecture notes, University of Malawi.

NET322: Network Programming & Applications Development – *AsyncIO in Python for Network Programming*, Lecture Notes, University of Malawi.

<https://www.youtube.com/watch?v=8aGhZQkoFbQ>

<https://www.youtube.com/watch?v=eiC58R16hb8>